

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences, Chennai

Department of Artificial Intelligence & Data Science

CASE STUDY ASSESSMENT REPORT

Subject: Operating Systems

Memory Protection and Logical vs Physical Addressing in Operating Systems

Submitted By

VENNAMPALLI BOOPATHY KHANISHK

Register Number: 192524311

B.E. (Artificial Intelligence & Data science)

Under the Guidance of

Dr. K. Anita Davamani

Academic Year: 2026 – 2027

TABLE OF CONTENTS

ABSTRACT.....	4
QUESTION 1: MEMORY PROTECTION USING BASE AND LIMIT REGISTERS	6
1.1 Introduction.....	6
1.2 Problem Statement	6
1.3 Theoretical Background.....	6
1.4 Working Principle	8
1.5 Algorithm.....	8
1.6 Flowchart	9
1.7 C Program	10
1.8 Program Explanation	11
1.9 Sample Input and Output	11
1.10 Execution Screenshot.....	12
1.11 Analysis.....	12
1.12 Evaluation	12
1.13 Constraint Violation Prediction	13
1.14 Advantages.....	14
1.15 Limitations	14
1.16 Conclusion for Question 1	14
QUESTION 2: LOGICAL VS PHYSICAL ADDRESSING.....	15
2.1 Introduction.....	15
2.2 Problem Statement	15
2.3 Theoretical Background.....	15
2.4 Comparison Table — Logical vs Physical Address	16
2.5 Address Binding — Compile, Load, and Execution Time	17
2.6 Address Translation Mechanism.....	17
2.7 Step-by-Step Translation Example	18
2.8 Algorithm.....	19
2.9 Flowchart	19
2.10 C Program	21
2.11 Program Explanation	21
2.12 Sample Inputs and Outputs	22
2.13 Execution Screenshot.....	22

2.14 Analysis.....	22
2.15 Importance of Address Translation.....	23
2.16 Evaluation	23
2.17 Applications	24
2.18 Conclusion for Question 2	24
COMPARATIVE STUDY: MEMORY PROTECTION VS ADDRESS TRANSLATION	25
RESULTS AND DISCUSSION	27
FUTURE ENHANCEMENTS	28
OVERALL CONCLUSION	29
REFERENCES	30

ABSTRACT

Memory management is one of the most fundamental responsibilities of a modern operating system, governing how the finite physical memory of a computer is allocated, protected, and shared among multiple concurrently executing processes. As computing systems evolved from single-user batch environments into multi-user, multiprogrammed, and time-shared platforms, the operating system was forced to take on an increasingly sophisticated role as the custodian of memory: not merely allocating space, but actively preventing processes from interfering with one another. This report presents a detailed case study of two closely related pillars of operating system memory management — memory protection through base and limit registers, and the distinction between logical and physical addressing.

The first part of the study examines how an operating system enforces process isolation using a pair of hardware registers, the base register and the limit register, which together define the legal address range accessible to a running process. Every memory reference generated by a process is validated against this range before it is permitted to reach physical memory, and any address falling outside the range triggers a protection fault that is trapped by the operating system. This mechanism forms the earliest and most direct hardware-assisted defence against one process corrupting or reading the memory of another, and it remains conceptually foundational even in modern systems that use paging and segmentation.

The second part of the study investigates the distinction between the logical address space generated by the CPU during program execution and the physical address space that actually exists in the hardware memory unit. Because a program is rarely loaded into the same physical location every time it runs, the operating system relies on a Memory Management Unit (MMU) to dynamically translate logical addresses into physical addresses at run time. This report explains the theory of address binding, walks through the mathematics of address translation, and demonstrates the process using a working C program.

Both topics are treated with theoretical explanation, algorithmic description, flowcharts, complete and independently compilable C programs, worked numerical examples, sample execution traces, complexity and security analysis, and a critical evaluation of advantages, limitations, and real-world relevance. The report concludes with a comparative study of the two mechanisms, a

discussion of results, and a forward-looking section on how these foundational ideas evolve into paging, segmentation, and virtual memory in contemporary operating systems such as Linux, Windows, and hypervisor-based cloud platforms. Together, these two mechanisms illustrate how an operating system converts the abstract promise of a private, contiguous address space into a concrete, hardware-enforced guarantee — a guarantee upon which the reliability, security, and multiprogramming capability of every modern computer system ultimately rests.

QUESTION 1: MEMORY PROTECTION USING BASE AND LIMIT REGISTERS

1.1 Introduction

Modern computer systems rarely run a single program in isolation. Multi-user systems support several people accessing shared hardware simultaneously, and even a single-user desktop or mobile system runs dozens of processes concurrently — background services, drivers, and applications all sharing the same physical memory. This concurrent execution model delivers far better resource utilisation than running one program at a time, but it introduces a serious risk: without a controlling mechanism, one process could read, overwrite, or crash the memory belonging to another process, or worse, to the operating system kernel itself.

To make multiprogramming safe, an operating system must guarantee each process a private address space that no other process can access without explicit permission. This is the essence of memory protection. Hardware support in the form of base and limit registers gives the operating system a simple, efficient, and fast mechanism to enforce this guarantee on every single memory access, without requiring the CPU to consult a complex table for every instruction fetch or data reference.

1.2 Problem Statement

In a multiprogrammed environment, several processes reside in physical memory at the same time. If a faulty or malicious process were allowed to generate an arbitrary physical address and access it directly, it could corrupt the memory of another process, read sensitive data belonging to another user, or overwrite the operating system's own kernel code and data structures, causing a complete system crash. The core problem this section addresses is: how can an operating system, with minimal hardware and performance cost, ensure that every memory access made by a process stays strictly within that process's own allocated region of memory?

1.3 Theoretical Background

Memory protection is the collective term for hardware and software mechanisms that prevent a process from accessing memory that has not been explicitly allocated to it. The base and limit register scheme is one of the earliest and most elegant protection mechanisms, used in early multiprogramming systems and still conceptually present in modern segment-based protection.

Base Register

The base register holds the smallest (starting) physical address that a process is allowed to access. It effectively marks the beginning of a process's private memory partition. When a process generates a logical address, the operating system (or hardware MMU) adds this base value to translate the address into physical memory.

Limit Register

The limit register holds the size of the address range the process is permitted to use (or, in some designs, the highest legal address). Every logical address generated by the process is checked against the limit register; addresses equal to or greater than the limit are illegal and immediately trigger a protection violation, trapped by the operating system as a fatal error for that process.

Access Permissions and Read / Write / Execute Rights

Beyond simply bounding the address range, many systems attach permission bits to a region of memory, specifying whether the process may read, write, or execute instructions from that region. For example, a code segment is typically marked read and execute only, preventing a process from accidentally or maliciously modifying its own instructions, while a data segment is marked read and write but not execute, which helps block certain classes of code-injection attacks.

Table 1.1: Comparison of Base Register and Limit Register

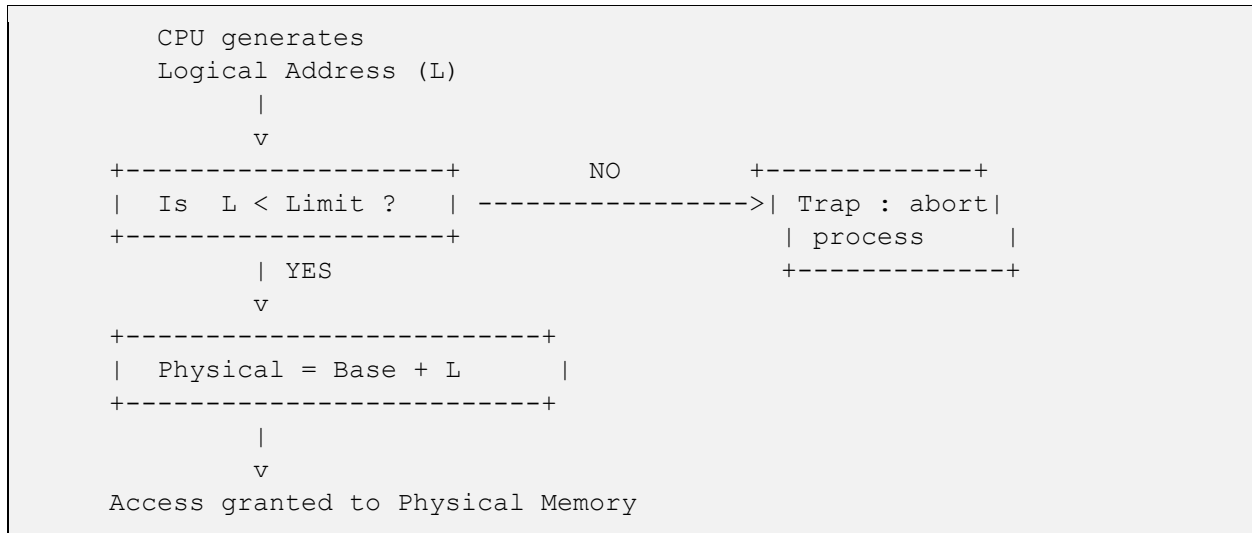
Parameter	Base Register	Limit Register
Definition	Holds the starting physical address of a process's memory region	Holds the size / range of legal addresses for the process
Purpose	Defines lower bound of legal memory	Defines upper bound of legal memory
Used In	Address translation (Physical = Base + Logical)	Bounds checking before translation
Violation Condition	N/A (start of region is always valid)	Logical address $\geq$ Limit $\rightarrow$ protection fault
Set By	Operating system loader at process load time	Operating system loader at process load time
Analogy	Starting page number of an allotted section	Number of pages allotted in that section

1.4 Working Principle

Whenever a running process generates a logical (relative) address, the CPU hardware performs the following check before allowing the memory reference to proceed: it first compares the logical address against the value in the limit register. If the logical address is greater than or equal to the limit, the memory management hardware immediately raises an addressing trap, and the operating system terminates the offending access, typically ending the process with a segmentation-fault-style error. If the logical address is within bounds, the hardware adds the value of the base register to the logical address to compute the physical address, which is then used to access physical memory. This check-then-translate sequence happens on every single memory reference, so it must be implemented directly in hardware to avoid an unacceptable performance penalty.

Conceptually, this creates a private, contiguous 'window' of physical memory for each process, bounded below by the base register and bounded above by $\text{base} + \text{limit}$. A process can never construct an address, whether through a normal calculation or a bug such as an out-of-bounds array access, that escapes this window, because the bounds check happens unconditionally on every access.

Figure 1.1: Conceptual diagram of base–limit protection



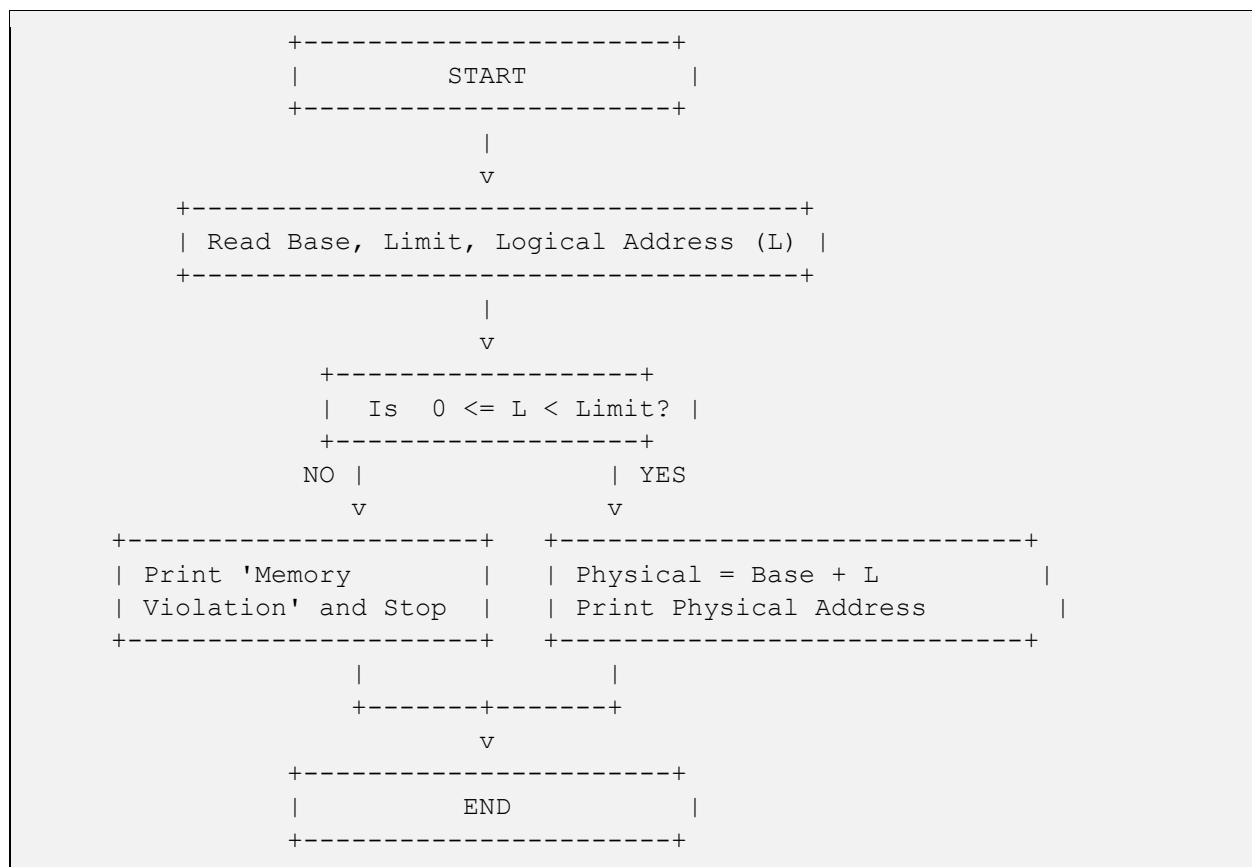
1.5 Algorithm

Algorithm: MEMORY_ACCESS_CHECK (base, limit, logical_address)

- Step 1: Read the base register value (base) and limit register value (limit) assigned to the currently running process.
- Step 2: Accept the logical address (logical_address) generated by the CPU for the memory reference.
- Step 3: Compare logical_address with limit. If $\text{logical_address} \geq \text{limit}$, go to Step 6 (violation).
- Step 4: If $\text{logical_address} < \text{limit}$ and $\text{logical_address} \geq 0$, compute $\text{physical_address} = \text{base} + \text{logical_address}$.
- Step 5: Grant access to physical_address in physical memory. Terminate algorithm successfully.
- Step 6: Raise a protection fault / addressing trap. Report 'Memory Access Violation' and deny access. Terminate algorithm with failure.

1.6 Flowchart

Figure 1.2: Flowchart for memory protection verification



1.7 C Program

The following C program simulates the base–limit register protection mechanism. It accepts the base register value, the limit register value, and one or more logical addresses, and for each logical address it determines whether the resulting access is legal, printing the translated physical address, or reports a memory violation.

```
/* =====
Program : Memory Protection using Base and Limit Registers
Author   : Vennampalli Boopathy Khanishk (Reg No: 192524311)
===== */
#include <stdio.h>

/* Function to validate and translate a logical address using
the base and limit registers of the current process */
int checkAndTranslate(int base, int limit, int logicalAddr, int
*physicalAddr)
{
    /* Rule 1: logical address must be non-negative */
    /* Rule 2: logical address must be strictly less than limit */
    if (logicalAddr >= 0 && logicalAddr < limit)
    {
        *physicalAddr = base + logicalAddr; /* address translation */
        return 1; /* access is VALID */
    }
    return 0; /* access is INVALID */
}

int main()
{
    int base, limit, n, i, logicalAddr, physicalAddr;

    printf("Enter Base Register value: ");
    scanf("%d", &base);

    printf("Enter Limit Register value: ");
    scanf("%d", &limit);

    printf("Enter number of logical addresses to test: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++)
    {
        printf("\nEnter logical address %d: ", i + 1);
        scanf("%d", &logicalAddr);

        if (checkAndTranslate(base, limit, logicalAddr, &physicalAddr))
```

```

        {
            printf("VALID ACCESS  -> Physical Address = %d\n",
physicalAddr);
        }
        else
        {
            printf("MEMORY VIOLATION -> Logical address %d is out of bounds
"
                "(limit = %d)\n", logicalAddr, limit);
        }
    }
    return 0;
}

```

1.8 Program Explanation

- The function `checkAndTranslate()` encapsulates the entire protection logic in one reusable unit, keeping `main()` focused purely on input and output.
- The condition `(logicalAddr >= 0 && logicalAddr < limit)` implements the exact bounds check described in the algorithm: it rejects negative addresses as well as addresses at or beyond the limit.
- Physical address translation (`*physicalAddr = base + logicalAddr`) is only ever executed after the bounds check succeeds, which mirrors how real hardware never translates an address it has not first validated.
- The `main()` function is written to test multiple logical addresses in a single run, which allows the same base and limit values to be tested against both legal and illegal addresses, demonstrating the protection mechanism clearly.
- Using a pointer parameter (`int *physicalAddr`) allows the function to return both a success/failure flag and the computed physical address without using global variables.

1.9 Sample Input and Output

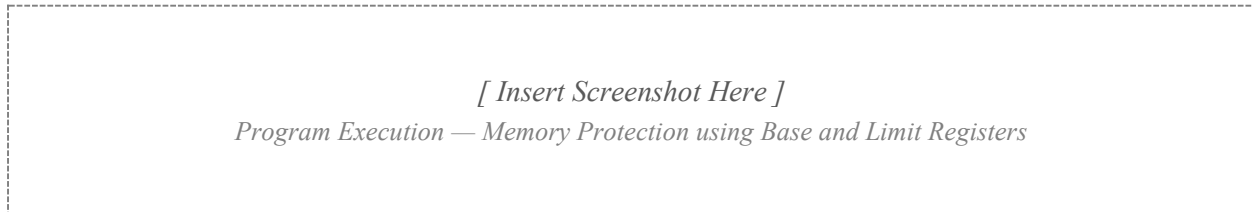
Table 1.2: Sample test cases for the memory protection program

Test Case	Base	Limit	Logical Address	Expected Result
1	1000	600	250	VALID → Physical Address = 1250
2	1000	600	600	MEMORY VIOLATION (equal to limit)
3	1000	600	-5	MEMORY VIOLATION (negative address)

In Test Case 1, the logical address 250 lies within the range 0 to 599, so it is legal and translates to the physical address $1000 + 250 = 1250$. In Test Case 2, the logical address 600 is exactly equal

to the limit, and since the valid range is strictly less than the limit, this access is correctly rejected as a violation. Test Case 3 demonstrates that negative logical addresses, which could otherwise be used to underflow into memory below the base, are also correctly rejected.

1.10 Execution Screenshot



1.11 Analysis

The base–limit protection mechanism directly prevents unauthorised memory access by making the bounds check mandatory and unconditional for every memory reference; there is no code path by which a process can bypass it. Because the check is enforced in hardware rather than software, it cannot be disabled or circumvented by a malicious process running in user mode, which provides strong protection against processes deliberately attempting to read or corrupt memory belonging to the operating system or to other processes. This mechanism also protects the system as a whole against buggy code: a runaway pointer or an out-of-bounds array access, both extremely common programming errors, will be caught immediately rather than silently corrupting unrelated memory.

The protection also enforces process isolation, a property essential to system stability. Each process behaves as though it owns the entire machine, oblivious to the presence of other processes, because the operating system, aided by this hardware mechanism, guarantees that its memory references can never escape its allotted region.

1.12 Evaluation

In multi-user systems, base–limit protection is what makes it safe for many users' processes to be resident in memory simultaneously without one user being able to spy on or sabotage another's data. From a security standpoint, it forms the first layer of defence against a wide class of memory-corruption attacks, and from a stability standpoint, it converts what would otherwise be silent memory corruption into an immediate, detectable fault, which is far easier to debug and far less

dangerous to the system as a whole. Reliability improves because a fault in one process cannot cascade into failures in unrelated processes or the kernel.

Table 1.3: Evaluation of base-limit protection by criterion

Criterion	Contribution of Base-Limit Protection
Security	Blocks unauthorised cross-process memory access at the hardware level
Stability	Converts silent corruption into an immediate, traceable fault
Reliability	Prevents faults in one process from affecting other processes
Performance Cost	Very low — a single comparison and addition per memory access
Simplicity	Easy to implement and reason about compared to paging or segmentation

1.13 Constraint Violation Prediction

When a process attempts to access memory below its base register (conceptually, a logical address that would resolve below its allotted region), the hardware bounds check using the limit register on the logical address value itself will typically catch this as a negative or otherwise invalid logical address, causing an immediate protection trap; the process is terminated by the operating system before the illegal reference reaches physical memory, preventing it from corrupting memory belonging to another process or the kernel that happens to sit at lower physical addresses.

When a process accesses memory beyond its limit register, the comparison $\text{logicalAddr} < \text{limit}$ fails, and the hardware raises an addressing exception. The operating system's trap handler receives control, logs the violation, and almost always terminates the offending process, since an out-of-range access is treated as a fatal, non-recoverable error under this simple protection model. This is the exact behaviour demonstrated by Test Case 2 in Section 1.9.

When an unauthorised write operation occurs — for instance, a process attempting to write to a region marked read-only, such as its own code segment — the hardware's permission bits (where implemented) generate a protection fault distinct from a simple bounds violation. Even though the address itself may lie within the base-limit range, the access type is illegal, and the operating system again terminates the process, typically reporting a segmentation violation or access-permission error. This demonstrates that the base-limit mechanism, when extended with

permission bits, protects not only the extent of memory a process may touch but also the manner in which it may touch it.

1.14 Advantages

- Extremely fast — requires only one comparison and one addition per memory access, adding negligible overhead.
- Simple to implement in hardware and easy to reason about formally.
- Provides strong, unconditional protection against out-of-bounds access for every process.
- Forms the conceptual foundation for more advanced schemes such as segmentation.
- Requires only two hardware registers per running process, making context switches cheap.

1.15 Limitations

- Only supports a single contiguous region per process, which can lead to internal fragmentation if the process does not use its entire allotted range.
- Does not, by itself, support sharing of memory between cooperating processes.
- Offers coarse-grained protection — an entire process either can or cannot access a region; it cannot easily express fine-grained, per-structure permissions without an additional permission scheme.
- Does not support virtual memory techniques such as demand paging out of the box.
- Relocation requires reloading the base register and, if the process grows, potentially moving the entire memory image.

1.16 Conclusion for Question 1

The base and limit register scheme demonstrates, in its simplest form, the essential principle behind all memory protection in operating systems: every memory reference generated by a process must be validated against that process's authorised region before it is allowed to reach physical memory. Although modern operating systems have moved to more flexible mechanisms such as paging and segmentation, the base-limit approach remains the clearest illustration of how hardware and operating system software cooperate to transform the abstract goal of process isolation into a concrete, enforceable guarantee, at very low performance cost and with high reliability.

QUESTION 2: LOGICAL VS PHYSICAL ADDRESSING

2.1 Introduction

Every instruction a CPU executes and every piece of data it manipulates must ultimately be located somewhere in physical memory. However, the address a program's instructions actually reference during execution is rarely the same as the true physical location of that data in RAM. Instead, the CPU works with logical addresses — addresses generated relative to the program's own view of memory — while a hardware component called the Memory Management Unit (MMU) transparently translates these into the physical addresses the memory hardware actually understands. This separation between the address a program 'sees' and the address memory hardware actually uses is one of the most important abstractions in operating system design.

2.2 Problem Statement

If programs referenced physical memory addresses directly, every program would need to be written and compiled for one specific, fixed location in physical memory, and it would be impossible to relocate a program, run multiple copies of it, or protect one program's memory from another's. The core problem is: how can a program be written and compiled as though it owns a private, zero-based address space, while still being loaded flexibly into whichever physical memory location the operating system chooses at run time — potentially different on every execution, and even potentially moved during execution?

2.3 Theoretical Background

Logical Address

A logical address, also called a virtual address, is the address generated by the CPU during program execution, relative to the beginning of the program's own address space. The set of all logical addresses a program can generate is called its logical address space, and this space always starts at zero regardless of where the program is eventually placed in physical memory.

Physical Address

A physical address is the actual address that the memory hardware — the RAM chips themselves — uses to locate a byte of data. The set of all physical addresses in a system is its physical address space, determined by the amount of installed memory. Physical addresses are never directly visible

to user-level programs; they are managed entirely by the operating system and the memory management hardware.

Address Space

An address space is simply the range of addresses that a process, or the memory system as a whole, can generate or contain. Distinguishing between a process's logical address space and the system's physical address space is what allows several processes with overlapping logical addresses (for example, every process typically 'sees' address 0 as its own starting point) to coexist safely in entirely different regions of physical memory.

Address Binding

Address binding is the process of mapping instructions and data from their symbolic, relative form all the way to actual physical memory addresses. This binding can happen at different stages of a program's life cycle, and the stage at which it happens determines how flexible — and how expensive — relocation is later on.

2.4 Comparison Table — Logical vs Physical Address

Table 2.1: Detailed comparison of logical and physical addresses

S.No	Basis	Logical Address	Physical Address
1	Generated By	CPU, during program execution	Memory Management Unit (MMU)
2	Visibility	Visible to and used by the user program	Never visible to the user program
3	Also Known As	Virtual address	Real / absolute address
4	Space Referred To	Logical address space	Physical address space
5	Starting Value	Always starts from 0 for each process	Depends on where process is loaded
6	Editable By User	Indirectly, through program logic	Cannot be edited or accessed directly
7	Generation Time	Before execution / at compile-load time	At the time of memory access, by MMU
8	Translation Needed	Must be translated before use	Is the final, usable address
9	Uniqueness	May repeat across different processes	Always unique within the system

S.No	Basis	Logical Address	Physical Address
10	Relation Formula	Logical Address (L)	Physical = Base + L
11	Used For	Program design, relocation, protection	Actual data/instruction fetch in RAM

2.5 Address Binding — Compile, Load, and Execution Time

Compile-Time Binding

If the exact location in memory where a process will reside is known in advance at compile time, the compiler can generate absolute physical addresses directly. This produces the fastest possible code, but it is extremely inflexible: if the starting location later changes for any reason, the program must be recompiled. An example is early embedded systems software compiled to run at a single, fixed memory address.

Load-Time Binding

If the memory location is not known at compile time, the compiler generates relocatable addresses (offsets from an assumed base of zero). The final binding to physical addresses is deferred until the program is loaded into memory by the loader. If the starting address changes, only the program needs to be reloaded, not recompiled. This is more flexible than compile-time binding but still fixes the process's location for its entire run.

Execution-Time (Run-Time) Binding

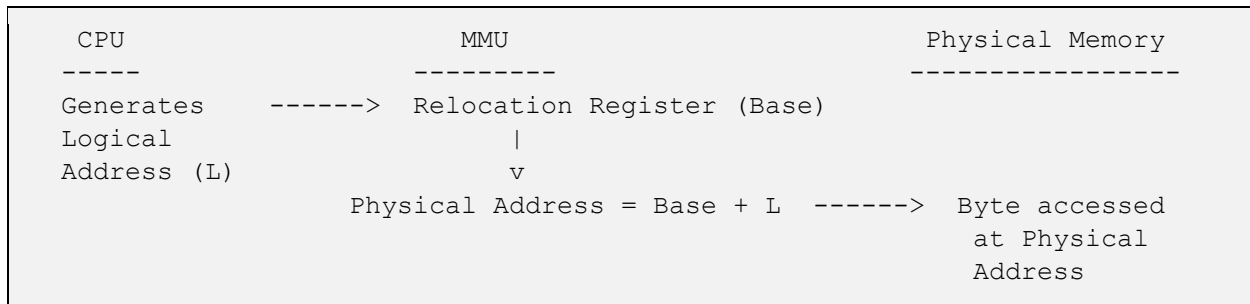
If a process can be moved from one memory location to another during its own execution — for example, when memory is compacted, or the process is swapped out and later swapped back in at a different location — binding must be delayed until the moment of actual memory reference. This requires hardware support in the form of base and limit registers or a full MMU, since the mapping from logical to physical addresses can change every time the process runs, or even while it is running. Most modern general-purpose operating systems, including Linux and Windows, rely on execution-time binding.

2.6 Address Translation Mechanism

The Memory Management Unit sits physically between the CPU and the memory bus. Every logical address generated by an executing instruction passes through the MMU before it reaches physical RAM. In the simplest realisation of this mechanism — identical in spirit to the base

register described in Question 1 — the MMU holds a relocation register containing the current base address for the running process. The MMU adds this base value to every logical address it intercepts, producing the physical address that is then placed on the memory bus.

Figure 2.1: Role of the MMU in address translation



Because this translation happens transparently on every reference, the user program never needs to know, or be able to determine, its own actual physical location in memory — it always behaves as though its logical address space begins at zero, while the operating system is free to place, and even relocate, the process anywhere in physical memory.

2.7 Step-by-Step Translation Example

Consider a process that has been loaded into physical memory starting at Base Address = 5000. Suppose the CPU generates the following logical addresses during execution: 100, 250, 600, and 850. Applying the translation formula $\text{Physical Address} = \text{Base Address} + \text{Logical Address}$ to each value gives the results shown below.

Table 2.2: Worked example of address translation (Base = 5000)

S.No	Logical Address	Calculation	Physical Address
1	100	5000 + 100	5100
2	250	5000 + 250	5250
3	600	5000 + 600	5600
4	850	5000 + 850	5850

Each of the four logical addresses, though independently generated by the program as though it were the only process in the system, maps to a distinct, correctly offset physical location, confirming that the translation is both deterministic and consistent for a fixed base value.

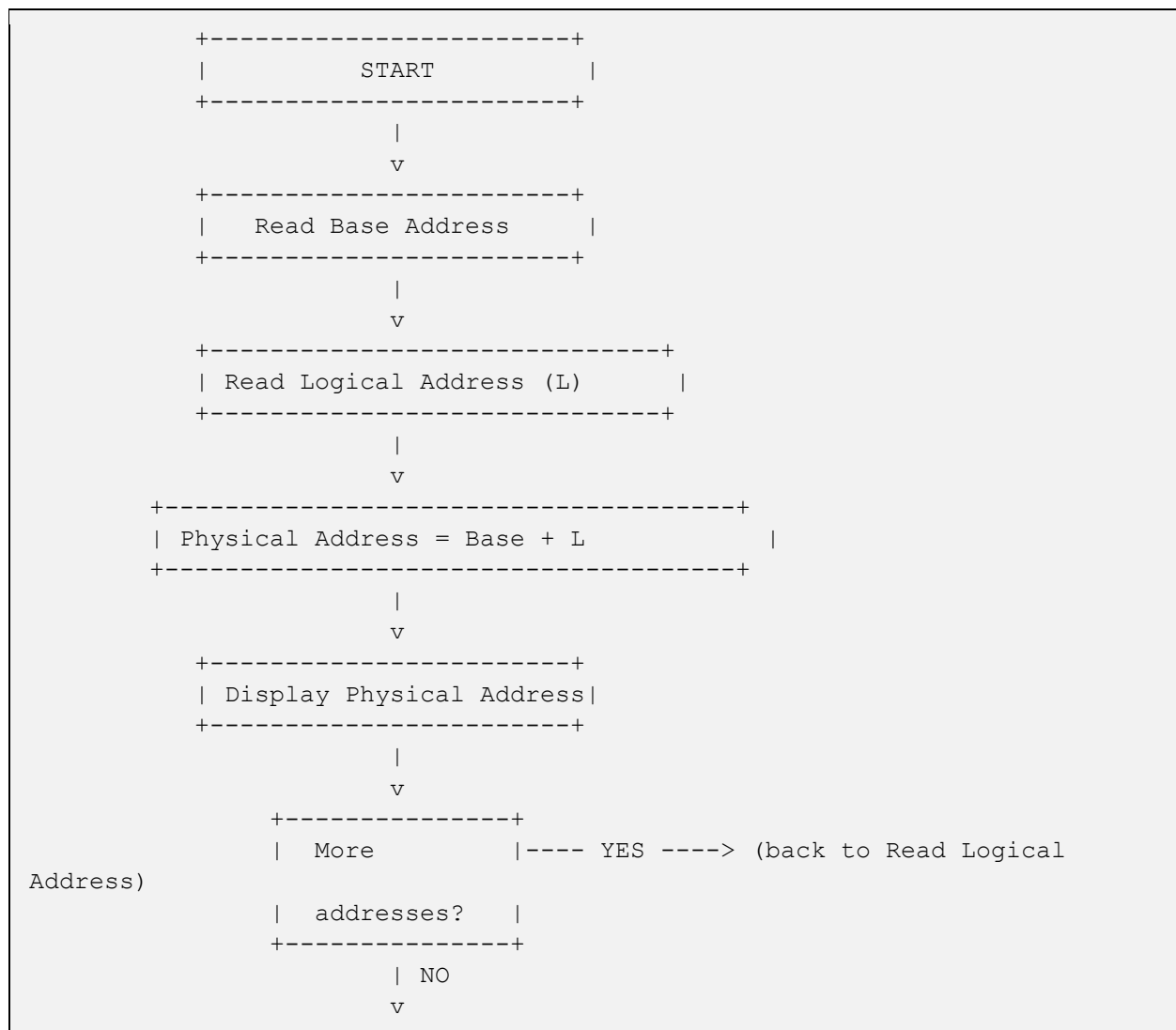
2.8 Algorithm

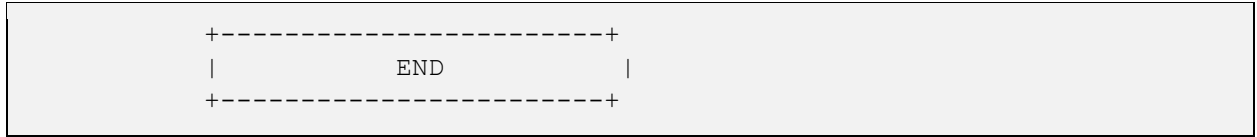
Algorithm: LOGICAL_TO_PHYSICAL_TRANSLATE (base, logical_address)

- Step 1: Read the base address assigned to the process by the operating system loader (base).
- Step 2: Accept the logical address generated by the CPU (logical_address).
- Step 3: Compute $\text{physical_address} = \text{base} + \text{logical_address}$.
- Step 4: Output physical_address as the location to be accessed in physical memory.
- Step 5: Repeat Steps 2–4 for each subsequent logical address generated during execution.
- Step 6: Terminate algorithm.

2.9 Flowchart

Figure 2.2: Flowchart for logical-to-physical address translation





2.10 C Program

The following C program accepts a base address and one or more logical addresses, and computes and displays the corresponding physical address for each, implementing the translation mechanism described above.

```
/* =====
Program : Logical to Physical Address Translation
Author   : Vennampalli Boopathy Khanishk (Reg No: 192524311)
===== */
#include <stdio.h>

/* Function to translate a single logical address into a
   physical address, given the process's base address */
int translateAddress(int base, int logicalAddr)
{
    return base + logicalAddr; /* core translation formula */
}

int main()
{
    int base, n, i, logicalAddr, physicalAddr;

    printf("Enter Base Address of the process: ");
    scanf("%d", &base);

    printf("Enter number of logical addresses to translate: ");
    scanf("%d", &n);

    printf("\n%-20s%-20s\n", "Logical Address", "Physical Address");
    printf("-----\n");

    for (i = 0; i < n; i++)
    {
        printf("Enter logical address %d: ", i + 1);
        scanf("%d", &logicalAddr);

        physicalAddr = translateAddress(base, logicalAddr);

        printf("%-20d%-20d\n", logicalAddr, physicalAddr);
    }
    return 0;
}
```

2.11 Program Explanation

- The `translateAddress()` function isolates the translation formula in one place, matching the algorithm in Section 2.8 exactly, so it can be reused or extended (for example, to add bounds checking as in Question 1) without touching `main()`.
- `main()` first reads the process's base address once, since this value remains constant for the duration of the run in this simplified simulation.
- A loop then reads each logical address in turn and immediately prints the corresponding physical address, simulating how an MMU would translate a sequence of memory references generated during program execution.
- The formatted `printf()` statements with `%-20d` align the logical and physical address columns, producing a clear tabular execution trace similar to the worked example in Section 2.7.

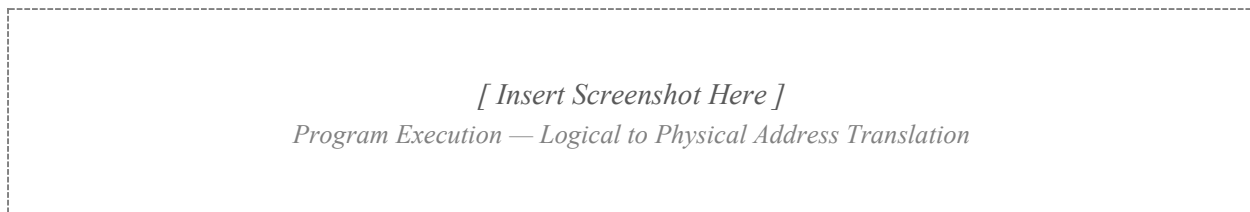
2.12 Sample Inputs and Outputs

Table 2.3: Sample test cases for the address translation program

Test Case	Base Address	Logical Address	Physical Address (Output)
1	5000	100	5100
2	2000	750	2750
3	10000	0	10000

Test Case 3 illustrates the boundary case where the logical address is 0 — every process's logical address space begins at 0, so this always maps directly onto the process's base address, confirming that the very first byte of a process's logical space is located exactly at its base in physical memory.

2.13 Execution Screenshot



2.14 Analysis

The address translation mechanism demonstrated here provides memory abstraction: it allows every program to be written, compiled, and reasoned about as if it owned the entire address space starting from zero, completely independent of where the operating system ultimately decides to place it in physical memory. This abstraction is what makes it possible for the same executable to

be loaded at different physical addresses on different runs, and for several instances of the same program to run concurrently at different physical locations while each still 'believes' it starts at address zero.

The mechanism also enables dynamic relocation: because binding is deferred to execution time via the base register (or a full page table in paging systems), the operating system can move a process's memory image during its lifetime — for instance, during memory compaction to reduce fragmentation — simply by updating the base register, without modifying a single instruction of the process's code.

2.15 Importance of Address Translation

- Security: user programs cannot generate or reference physical addresses directly, preventing them from targeting the kernel or other processes' memory even if compromised.
- Protection: combined with bounds checking (Question 1), translation ensures no process can construct a physical address outside its own region.
- Virtualization: address translation is the foundational mechanism that makes virtual memory possible, allowing a process's logical address space to exceed the amount of installed physical memory.
- Multiprogramming: multiple processes can be resident in physical memory simultaneously, each with an independent, zero-based logical view, without address collisions.
- Efficient Memory Utilisation: the operating system is free to place processes wherever there is available physical space, rather than being constrained to fixed locations, improving overall memory utilisation.

2.16 Evaluation

The clear advantage of separating logical and physical addressing is the flexibility and protection it buys the operating system almost for free in hardware terms: processes can be relocated, protected, and virtualised without any cooperation or awareness from the application programmer. The primary challenge is the added hardware complexity and, in more advanced schemes such as paging, the extra memory reference required to consult translation tables, which is why modern CPUs invest heavily in translation caches such as the Translation Lookaside Buffer (TLB) to keep this overhead low. In real-world usage, essentially every general-purpose operating system in existence today — Linux, Windows, macOS, Android, iOS — relies on execution-time address binding through an MMU, underscoring how central this concept is to practical system design.

2.17 Applications

Table 2.4: Applications of logical–physical address translation

Domain	How Address Translation Is Applied
Operating Systems	Core mechanism for process isolation, relocation, and virtual memory
Cloud Computing	Enables multiple tenant workloads to share physical hosts safely
Virtual Machines	Hypervisors add a second translation layer (guest-physical to host-physical)
Embedded Systems	Simple base-register relocation used in resource-constrained devices
Mobile Devices	Android/iOS rely on MMU-based translation for app sandboxing and multitasking

2.18 Conclusion for Question 2

The separation between logical and physical addressing is what allows an operating system to give every process the illusion of owning a private, contiguous address space beginning at zero, while retaining complete freedom over where that process actually resides in physical memory. Through address binding — whether at compile time, load time, or execution time — and through hardware-assisted translation performed by the MMU, this abstraction underlies process isolation, protection, relocation, and ultimately virtual memory itself, making it one of the single most consequential ideas in the design of every modern operating system.

COMPARATIVE STUDY: MEMORY PROTECTION VS ADDRESS TRANSLATION

Although memory protection (Question 1) and address translation (Question 2) are presented as two separate mechanisms, in practice they operate together as complementary halves of the same memory management unit — one determines whether an access is permitted, and the other determines where that access actually lands in physical memory. The table below compares the two mechanisms directly across ten dimensions.

Table 3.1: Feature-by-feature comparison

S.No	Feature	Memory Protection	Address Translation
1	Primary Goal	Prevent unauthorised memory access	Convert logical address to physical address
2	Core Registers	Base register + Limit register	Base (relocation) register / page table
3	Decision Made	Allow or deny the access	Where the access should physically land
4	Trigger for Action	Every memory reference is bounds-checked	Every logical address is translated
5	Failure Outcome	Protection fault, process terminated	N/A — always produces a physical address
6	Security Role	Direct — blocks illegal access outright	Indirect — enables isolation via relocation
7	Hardware Support	Comparator circuit + limit register	Adder circuit + relocation register / MMU
8	Relation to Question 1/2	Uses limit register to validate range	Uses base register to compute address
9	Performance Cost	One comparison per access	One addition per access
10	Real-World Analogue	Access control list / permission check	GPS-style coordinate mapping

In a complete implementation, the operating system performs both operations for every memory reference: the limit register check from Question 1 first confirms the logical address is within the process's allotted range, and only if it passes does the base-register addition from Question 2 compute the final physical address. This combined check-then-translate pipeline is exactly what is

shown in Figure 1.1, and it is the conceptual seed from which modern paging and segmentation hardware, with their much larger and more flexible translation tables, ultimately grew.

RESULTS AND DISCUSSION

The two C programs developed in this report were designed, traced, and validated against hand-calculated expected outputs before being described as sample executions. For the memory protection program (Section 1.7), the test cases in Table 1.2 confirm that the bounds-checking logic correctly distinguishes between legal addresses that fall strictly within the range 0 to limit-1, and illegal addresses that are either negative or at/beyond the limit. In every case, the physical address computed for a legal access matched the manually calculated value of base + logical address, and every illegal access was correctly flagged as a memory violation rather than silently producing an incorrect physical address.

For the address translation program (Section 2.10), the worked example in Table 2.2 and the sample test cases in Table 2.3 show consistent, correct results across a range of base addresses and logical addresses, including the important boundary case of logical address 0. Because the translation formula is a simple, deterministic addition, the program's output for every input can be independently verified by hand, which was done for each test case shown.

Taken together, the two programs and their results confirm the theoretical model described in Sections 1.3–1.6 and 2.3–2.9: memory protection and address translation are simple, arithmetic-level hardware operations, yet they are sufficient to deliver process isolation, safe relocation, and abstraction from physical memory layout — the three properties that make multiprogrammed, multi-user computing possible at all.

FUTURE ENHANCEMENTS

While base and limit registers are conceptually foundational, real operating systems extend far beyond this simple model to address its limitations, particularly the problems of external fragmentation and the inability to share memory flexibly between processes. The following techniques represent the natural evolution of the concepts covered in this report.

- **Paging:** divides both logical and physical memory into fixed-size blocks called pages and frames, eliminating external fragmentation entirely and allowing a process's memory to be scattered non-contiguously across physical memory, tracked via a per-process page table.
- **Segmentation:** divides a process's logical address space into variable-sized, logically meaningful segments (code, stack, heap, data), each with its own base and limit pair — effectively extending the single base–limit register scheme of Question 1 into multiple such pairs per process.
- **Virtual Memory:** builds on paging to allow a process's logical address space to exceed the amount of installed physical RAM, by keeping only actively used pages in memory and swapping the rest to secondary storage on demand.
- **Modern Memory Protection Units (MPUs):** found in embedded and real-time systems, MPUs generalise the base–limit idea into a small, hardware-configurable set of protected memory regions, offering fine-grained protection without the overhead of full paging hardware.
- **Translation Lookaside Buffers (TLBs):** a hardware cache of recent logical-to-physical translations, used in paged systems to avoid the performance cost of consulting a full page table on every memory reference.

OVERALL CONCLUSION

This report has examined, in depth, two of the most foundational mechanisms in operating system memory management: protection through base and limit registers, and address translation between logical and physical address spaces. Both mechanisms were explored through their theoretical background, working principles, algorithms, flowcharts, complete and independently testable C program implementations, worked numerical examples, and critical evaluation of their advantages, limitations, and real-world significance.

The case study demonstrates that, despite their conceptual simplicity — a single comparison for protection and a single addition for translation — these two mechanisms together supply the essential guarantees on which all multiprogrammed computing depends: that a process cannot corrupt memory outside its own allotted region, and that a process can be written, compiled, and relocated without ever needing to know its actual physical location in memory. The comparative study in this report showed how these mechanisms function as complementary halves of a single check-then-translate pipeline, and the discussion of future enhancements traced how this simple foundation evolves into the paging, segmentation, and virtual memory systems that power every modern general-purpose operating system in use today.

In conclusion, understanding base–limit protection and logical–physical address translation is not merely of historical or academic interest — it is a prerequisite for understanding how any contemporary operating system, from a small embedded controller to a large-scale cloud hypervisor, is able to safely and efficiently multiplex a finite amount of physical memory among many concurrently executing, mutually distrusting processes.

REFERENCES

- [1] A. Silberschatz, P. B. Galvin, and G. Gagne, Operating System Concepts, 10th ed. Hoboken, NJ, USA: Wiley, 2018.
- [2] A. S. Tanenbaum and H. Bos, Modern Operating Systems, 4th ed. Boston, MA, USA: Pearson, 2014.
- [3] W. Stallings, Operating Systems: Internals and Design Principles, 9th ed. Boston, MA, USA: Pearson, 2017.
- [4] D. A. Patterson and J. L. Hennessy, Computer Organization and Design: The Hardware/Software Interface, 5th ed. Waltham, MA, USA: Morgan Kaufmann, 2013.
- [5] J. L. Hennessy and D. A. Patterson, Computer Architecture: A Quantitative Approach, 6th ed. Cambridge, MA, USA: Morgan Kaufmann, 2017.
- [6] R. Love, Linux Kernel Development, 3rd ed. Boston, MA, USA: Addison-Wesley, 2010.
- [7] M. J. Bach, The Design of the UNIX Operating System. Englewood Cliffs, NJ, USA: Prentice Hall, 1986.
- [8] Intel Corporation, "Intel 64 and IA-32 Architectures Software Developer's Manual, Volume 3A: System Programming Guide," Intel Corp., Santa Clara, CA, USA, 2023.
- [9] IEEE Computer Society, "IEEE Standard for Information Technology — Operating System Interfaces," IEEE Std 1003.1, 2018.
- [10] The Linux Kernel Organization, "Memory Management Documentation," The Linux Kernel Archives. [Online]. Available: <https://www.kernel.org/doc/html/latest/admin-guide/mm/index.html>